

SAPIENZA
UNIVERSITÀ DI ROMA

"Sapienza" University of Rome
Faculty of Information Engineering,
Informatics and Statistics
Department of Computer Science

FDS

Fundamentals of Data Science

Author
Matteo

October 11, 2026

Contents

1 Introduction **1**

1.1 What is Data Science? 1

1.2 The Data Science lifecycle 2

1.3 Artificial Intelligence and Machine Learning 3

1.4 Supervised models and prediction tasks 4

2 Data Representation **5**

2.1 The Iris dataset 5

2.2 Features and targets 6

2.3 Data types 6

2.4 Encoding categorical features 7

2.5 Labels 8

2.6 Dataset notation 9

2.7 Data generation and the i.i.d. assumption 10

2.8 Data points as vectors 11

2.9 Spanning sets 12

2.10 Basis 13

2.11 Standard basis 15

2.12 Coordinates in an orthonormal basis 18

2.13 Nonstandard bases 20

2.14 Finding the weights by least squares 21

2.15 Using fewer directions: subspaces 27

2.16 Projection onto a subspace 28

2.17 Orthonormal directions in a subspace 31

3 Principal Component Analysis **34**

4 Optimization **35**

4.1 What are we trying to optimize? 35

4.2 Global minima and maxima 36

4.3 Local minima and maxima 38

4.4 Why these are called zero-order conditions 39

1 Introduction

Start of lecture 00 (Spinelli)

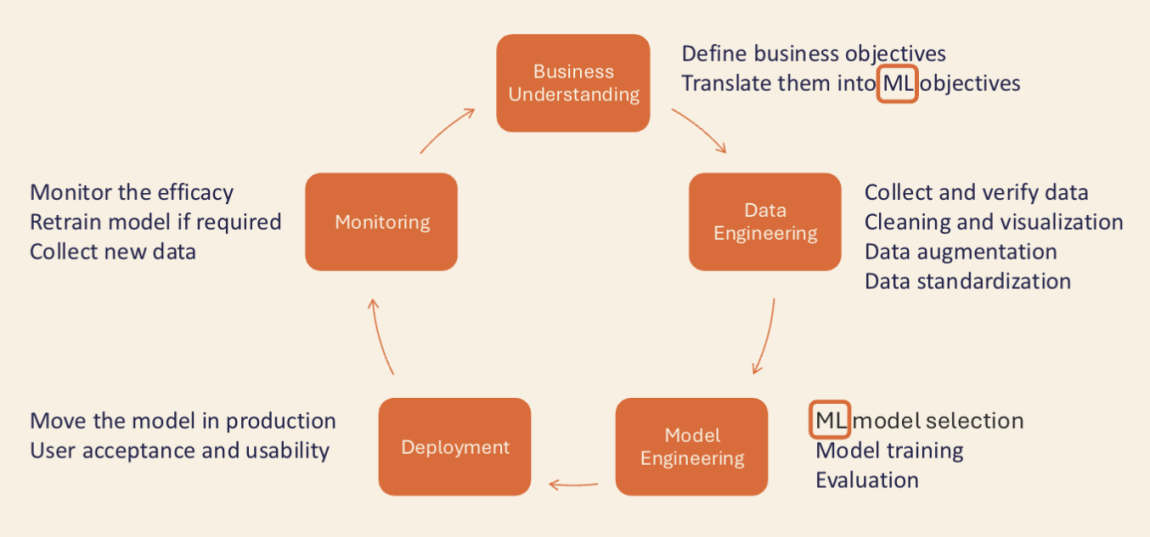
1.1 What is Data Science?

Definition 1.1: Data Science

Data Science is an interdisciplinary field that extracts knowledge from data to support decisions. It combines statistics, mathematics, computer science, and domain expertise to collect, clean, manage, and analyse data.

1.2 The Data Science lifecycle

A project moves through business understanding, data engineering, model engineering, deployment, and monitoring. This cycle can repeat when new data or retraining is needed.



The Data Science lifecycle. Source: Spinelli, Lecture 00.

1.3 Artificial Intelligence and Machine Learning

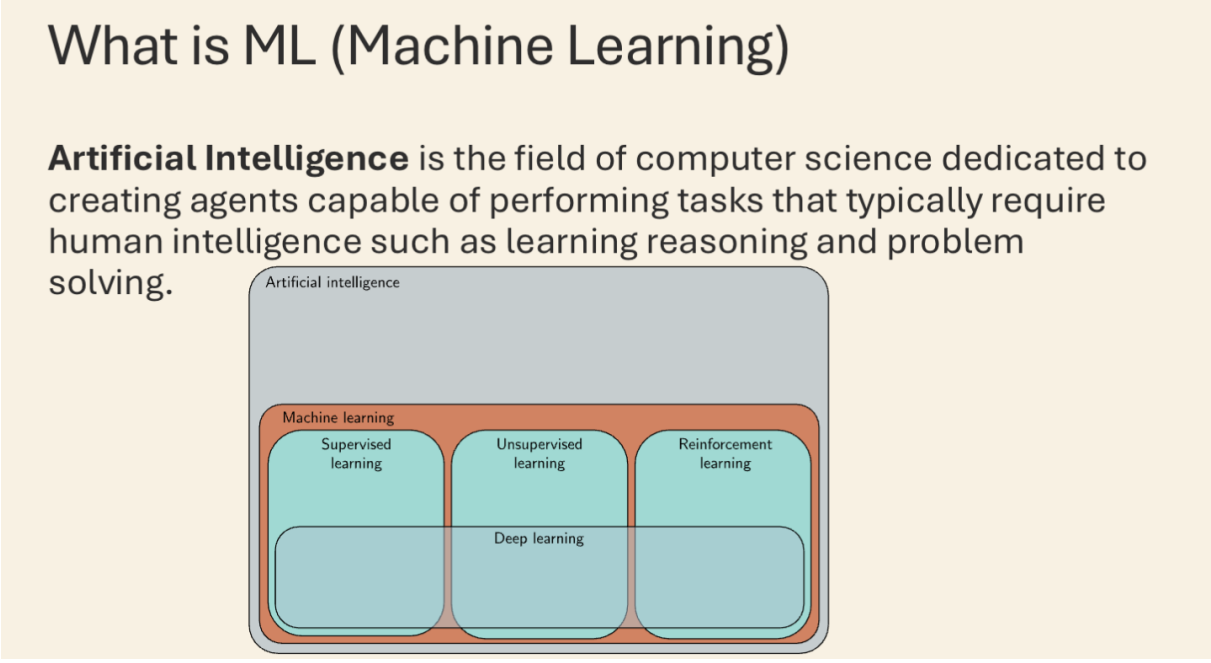
Artificial Intelligence (AI) studies how to build systems that perform tasks requiring abilities such as reasoning, learning, and problem solving.

Definition 1.2: Machine Learning

Machine Learning (ML) studies algorithms whose performance P on a task T improves through experience E. A learning problem therefore specifies the task, the performance measure, and the available experience.

The main learning settings are:

- **Supervised learning:** learn from inputs paired with known desired outputs to predict the output for new inputs.
- **Unsupervised learning:** discover structure in data without given target outputs.
- **Reinforcement learning:** learn how to act through interaction with an environment and feedback in the form of rewards.



AI, ML, and learning settings. Source: Spinelli, Lecture 00.

Deep learning uses neural networks with multiple layers and can be applied in each of these settings.

1.4 Supervised models and prediction tasks

Definition 1.3: Supervised learning model

A supervised learning model is a function $f : \mathcal{X} \to \mathcal{Y}$ that maps an input x to a predicted output $\hat{y} = f(x)$. Here, $\mathcal{X}$ is the input space and $\mathcal{Y}$ is the output space. Training means choosing or adjusting f using examples of inputs paired with their known outputs.

Definition 1.4: Regression

Regression is a supervised learning task whose target is a real-valued number or vector. For example, predicting a person's height from their age is a regression task.

Definition 1.5: Classification

Classification is a supervised learning task whose target is a category from a finite set of classes. For example, predicting whether a flower is setosa, versicolor, or virginica is a classification task.

2

Data Representation

Start of lecture 01 (Spinelli) — 2 October 2026

2.1 The Iris dataset

The Iris dataset contains 150 flowers, divided into three species: 50 setosa, 50 versicolor, and 50 virginica. The goal is to predict a flower's species from its measurements.

Example 1: Iris as a supervised classification task

Each **observation** is one flower, described by four **features**, all measured in centimetres:

- sepal length and sepal width;
- petal length and petal width.

The **target** is the flower's species. This is supervised learning because the species is known for the training examples, and classification because the target is a category.

2.2 Features and targets

In a tabular dataset, each row describes one observation. The feature columns contain its input measurements, while the target column contains the output we want to predict.

For the first flower shown in the lecture's table,

$$x = (4.3,\ 3.0,\ 1.1,\ 0.1), \qquad y = \text{setosa}.$$

The entries of x are, in order, sepal length, sepal width, petal length, and petal width, all in centimetres.

During training, both x and y are known. For a new input x, the model produces a prediction $\hat{y} = f(x)$. The observed target y and the predicted target $\hat{y}$ may differ.

2.3 Data types

Features and targets can have different types:

Type	**Values**	**Example**
Real-valued	$\mathbb{R}$	Petal length
Integer-valued	$\mathbb{Z}$	Number of purchases
Categorical	$\{C_1, \ldots, C_g\}$	Flower species
Binary	$\{0, 1\}$	Spam or non-spam

Predicting a numerical quantity is a regression task; predicting a category is a classification task. In Iris, the four features are numerical measurements and the target has three categories.

Assigning codes $0, 1, 2$ to the species does not turn them into numerical quantities: the codes identify categories, so the task remains classification.

2.4 Encoding categorical features

How can we give a colour to a model that works with numbers? Many learning algorithms require numerical features, so we need an *encoding*: a way to represent a category using numbers.

We could assign the codes red = 0, green = 1, and blue = 2. But a model that treats these codes as quantities could interpret blue as greater than green, or as twice as far from red as green is. Colours have no such order, and these numerical distances have no meaning here.

One-hot encoding replaces the colour column with three yes/no questions: "Is it red?", "Is it green?", and "Is it blue?". We use 1 for yes and 0 for no.

Example 2: Encoding a colour feature

Colour	Is it red?	Is it green?	Is it blue?
Red	1	0	0
Green	0	1	0
Blue	0	0	1

For example, a green product is represented by $(0, 1, 0)^\top$. Exactly one entry is 1 and all the others are 0: this is why the encoding is called *one-hot*.

A model predicting a product's price can then learn a separate effect for each colour, without assuming that blue is "twice" green. The cost is more columns: one feature with k categories becomes k numerical columns. We can now write this idea formally.

Definition 2.1: One-hot encoding

For a feature with k mutually exclusive categories, one-hot encoding represents category C_i by a vector $e_i \in \{0, 1\}^k$ whose i-th entry is 1 and whose other entries are 0:

$$(e_i)_j = \begin{cases} 1, & j = i, \\ 0, & j \neq i. \end{cases}$$

Each observation belongs to exactly one category, so its encoded vector has exactly one nonzero entry.

When categories have a natural order, an *ordinal encoding* can preserve it. For example, low, medium, and high can be coded as $0, 1, 2$. This expresses their order; it does not establish that the gaps between successive categories are equal.

2.5 Labels

Definition 2.2: Labels and labeled data

A *label* is an observed target value. In *labeled data*, the input and its target are both available.
In *unlabeled data*, the input is available but its target is unknown or unrecorded.

In Iris, a flower with measurements and a known species is labeled; the same measurements without the species are unlabeled. A label can also be numerical, as in a regression task.

2.6 Dataset notation

Let n be the number of observations and p the number of numerical features after any encoding. Write $\mathcal{X} \subseteq \mathbb{R}^p$ for the input space and $\mathcal{Y}$ for the target space.

Definition 2.3: Supervised dataset

A labeled dataset is an indexed collection of input–target pairs,

$$D = \big((x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\big) \in (\mathcal{X} \times \mathcal{Y})^n.$$

The i-th observation has an input column vector

$$x^{(i)} = (x_1^{(i)}, \ldots, x_p^{(i)})^\top \in \mathbb{R}^p$$

and a target $y^{(i)} \in \mathcal{Y}$.

The superscript (i) indexes an observation; the subscript j indexes a feature. Across all observations, the j-th feature column is

$$\mathbf{x}_j = (x_j^{(1)}, \ldots, x_j^{(n)})^\top \in \mathbb{R}^n.$$

Thus an input vector contains p features of one observation, while a feature column contains n values of one feature.

Example 3: A small supervised dataset

Suppose we predict house prices from area and room count. Here is a toy dataset:

House i	Area (m^2)	Rooms	Price (EUR)
1	50	2	150 000
2	80	3	240 000
3	100	4	300 000

Here $n = 3$ houses and $p = 2$ input features. The price is the target, so it is not counted in p. For the first house,

$$x^{(1)} = (50, 2)^\top, \qquad y^{(1)} = 150\,000.$$

D collects the three input–target pairs. Here $x_2^{(1)} = 2$ is the first house's room count. The rooms column is

$$\mathbf{x}_2 = (2, 3, 4)^\top.$$

The vector $x^{(1)}$ describes one house using both features; the vector $\mathbf{x}_2$ contains one feature for all three houses.

2.7 Data generation and the i.i.d. assumption

We model the observations as realizations of random pairs (X, Y) with an unknown joint probability distribution P_{XY} on $\mathcal{X} \times \mathcal{Y}$. Capital letters denote random variables; lowercase letters denote their observed values. Supervised learning uses these observations to learn a predictive relationship between input and target.

Definition 2.4: I.i.d. observations

The observations are *i.i.d.* when their random pairs $(X^{(1)}, Y^{(1)}), \ldots, (X^{(n)}, Y^{(n)})$ are mutually independent and all have the same distribution P_{XY}.

Independence is between observations; it does not require X and Y within one observation to be independent. The i.i.d. assumption can fail, for example when successive observations form a time series.

2.8 Data points as vectors

We now focus on the inputs $x^{(i)} \in \mathbb{R}^p$. This space supports vector addition and multiplication by real numbers, called *scalars*. These operations act coordinate by coordinate:

$$u + v = (u_1 + v_1, \ldots, u_p + v_p)^\top, \qquad au = (au_1, \ldots, au_p)^\top.$$

An input can be drawn as a point with those coordinates or as a *position vector*: an arrow from the origin to that point.

2.8.1 Mean-centering

Definition 2.5: Mean-centered data

The sample mean input vector and the centered observations are

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x^{(i)}, \qquad \widetilde{x}^{(i)} = x^{(i)} - \bar{x}.$$

The mean of the centered inputs is zero: $\frac{1}{n} \sum_{i=1}^{n} \widetilde{x}^{(i)} = 0$.

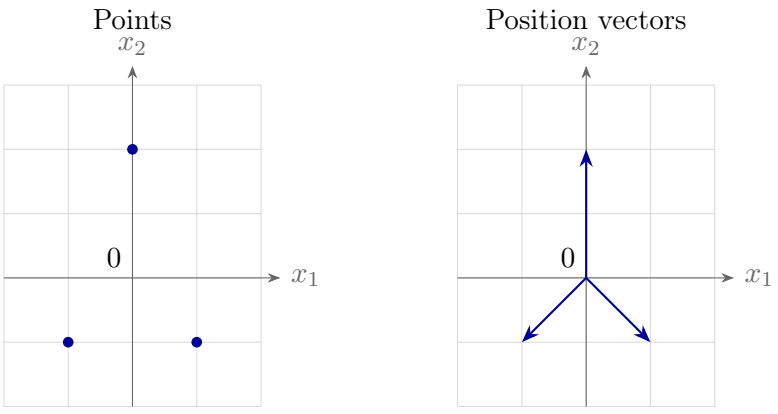
Centering subtracts each feature's sample mean from that feature. It is reversible if we retain $\bar{x}$, since $x^{(i)} = \widetilde{x}^{(i)} + \bar{x}$.

Example 4: Centering three points

For the inputs $(1, 1)^\top$, $(3, 1)^\top$, and $(2, 4)^\top$,

$$\bar{x} = \left(\frac{1+3+2}{3}, \frac{1+1+4}{3} \right)^\top = (2, 2)^\top.$$

The centered inputs are $(-1, -1)^\top$, $(1, -1)^\top$, and $(0, 2)^\top$. Their sum is $(0, 0)^\top$.



The same centered observations, drawn as points and as arrows.

2.9 Spanning sets

Definition 2.6: Span and spanning set

The *span* of vectors $c_1, \ldots, c_K$ is the set of all their linear combinations:

$$\operatorname{span}\{c_1, \ldots, c_K\} = \left\{ \sum_{j=1}^{K} a_j c_j : a_1, \ldots, a_K \in \mathbb{R} \right\}.$$

These vectors form a *spanning set* of a vector space V when their span equals V: every vector in V can be expressed using them.

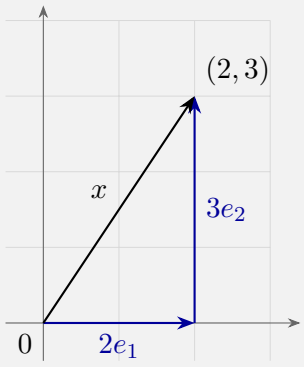
Example 5: Spanning the plane

The vectors

$$e_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad e_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

span $\mathbb{R}^2$, because every $(a, b)^\top$ can be written as $ae_1 + be_2$. For example,

$$\begin{pmatrix} 2 \\ 3 \end{pmatrix} = 2e_1 + 3e_2.$$



Using only e_1 produces vectors $(a, 0)^\top$, so it spans the horizontal axis rather than the whole plane.

2.10 Basis

Definition 2.7: Linear independence

Vectors $c_1, \ldots, c_K \in \mathbb{R}^p$ are *linearly independent* if the only scalars $a_1, \ldots, a_K \in \mathbb{R}$ satisfying

$$a_1 c_1 + \cdots + a_K c_K = 0$$

are $a_1 = \cdots = a_K = 0$.

Definition 2.8: Basis

A *basis* of a vector space is a linearly independent spanning set. Equivalently, it is a minimal spanning set: removing any of its vectors destroys the spanning property.

Example 6: A spanning set with a redundant vector

The pair $\{e_1, e_2\}$ is a basis of $\mathbb{R}^2$. Adding $c_3 = e_1 + e_2$ still gives a spanning set, but it is not a basis, since

$$e_1 + e_2 - c_3 = 0$$

is a linear combination with nonzero coefficients. The third vector is redundant because the first two already generate it.

2.10.1 Representing observations in a basis

A basis of $\mathbb{R}^p$ contains exactly p vectors, each in $\mathbb{R}^p$. For a basis $c_1, \ldots, c_p$, every input observation has a unique representation

$$x^{(i)} = \sum_{j=1}^{p} w_j^{(i)} c_j, \qquad i = 1, \ldots, n.$$

The coefficients $w_j^{(i)}$ are the *weights*, or coordinates, of that observation in the chosen basis. The vector $w^{(i)} = (w_1^{(i)}, \ldots, w_p^{(i)})^\top$ records its representation; combining the basis vectors with these weights reconstructs $x^{(i)}$.

Example 7: Coordinates in a basis

Use the same basis for both observations:

$$c_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

The two points have different weights:

$$x^{(1)} = \begin{pmatrix} 2 \\ 3 \end{pmatrix} = 2c_1 + 3c_2,$$

$$x^{(2)} = \begin{pmatrix} 4 \\ 1 \end{pmatrix} = 4c_1 + c_2.$$

Observation	**Weight of c_1**	**Weight of c_2**
First	$w_1^{(1)} = 2$	$w_2^{(1)} = 3$
Second	$w_1^{(2)} = 4$	$w_2^{(2)} = 1$

For example, $w_2^{(1)} = 3$ means that the first observation uses three times the second basis vector. The superscript (i) identifies the observation; the subscript j identifies the basis vector.

2.11 Standard basis

Definition 2.9: Standard basis

The *standard basis* of $\mathbb{R}^p$ consists of the vectors $c_1, \ldots, c_p$, where c_j has a 1 in position j and zeros everywhere else. In $\mathbb{R}^2$,

$$c_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Each vector points along one coordinate axis. Consequently, the weights in this basis are the entries of the observation itself:

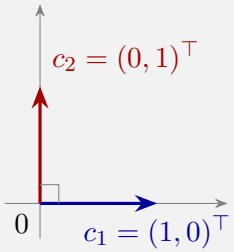
$$x^{(i)} = x_1^{(i)} c_1 + \cdots + x_p^{(i)} c_p \qquad \Longrightarrow \qquad w^{(i)} = x^{(i)}.$$

Example 8: Reading the weights from the coordinates

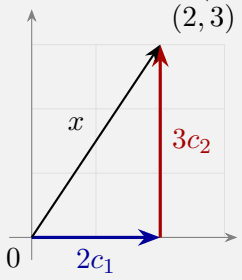
For $x = (2, 3)^\top$, move two units along the first axis and three along the second:

$$x = 2c_1 + 3c_2, \qquad w = \begin{pmatrix} 2 \\ 3 \end{pmatrix} = x.$$

Unit basis vectors



Representing $x = (2, 3)^\top$



The blue arrow contributes the first coordinate; the red arrow contributes the second. Their sum is the black arrow.

2.11.1 Orthogonality, unit length, and the basis matrix

The standard basis has two properties:

- **Different vectors are perpendicular:** $c_j^\top c_k = 0$ for $j \neq k$.
- **Each vector has unit length:** $\|c_j\|_2^2 = c_j^\top c_j = 1$.

In $\mathbb{R}^2$, these follow directly from

$$c_1^\top c_2 = 1 \cdot 0 + 0 \cdot 1 = 0, \qquad \|c_1\|_2^2 = 1^2 + 0^2 = 1,$$

and similarly $\|c_2\|_2^2 = 1$.

Definition 2.10: Orthonormal basis

A basis is *orthonormal* if its vectors are mutually perpendicular and each has length 1.

How can we check whether a basis is orthonormal? We need to check the length of every vector and whether each pair of different vectors is perpendicular. With many vectors, drawing arrows is impractical. We can collect all these checks in one matrix.

First, place the basis vectors in the columns of C:

$$C = [\, c_1 \ \cdots \ c_p \,] \in \mathbb{R}^{p \times p}.$$

Now compute $C^\top C$. Each entry is a dot product between two columns of C: the entry in row j, column k is $c_j^\top c_k$. We read the result as a table of checks:

Position	What it tells us	Required value
On the diagonal	A vector's length squared	1
Off the diagonal	The dot product of two different vectors	0

If the diagonal contains only 1s and every other entry is 0, both checks pass: the basis is orthonormal. That result is the identity matrix I_p, so the complete test is

$$\boxed{C^\top C = I_p.}$$

This formula is a compact way to check the lengths and perpendicularity of all the basis vectors at once. The standard basis passes the test.

Example 9: Using the orthonormality test

Consider the basis

$$c_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 0.6 \\ 0.8 \end{pmatrix}, \qquad C = \begin{pmatrix} 1 & 0.6 \\ 0 & 0.8 \end{pmatrix}.$$

Both vectors have length 1, since $1^2 + 0^2 = 1$ and $0.6^2 + 0.8^2 = 1$. Do they also pass the perpendicularity check?

$$C^\top C = \begin{pmatrix} 1 & 0 \\ 0.6 & 0.8 \end{pmatrix} \begin{pmatrix} 1 & 0.6 \\ 0 & 0.8 \end{pmatrix} = \begin{pmatrix} 1 & 0.6 \\ 0.6 & 1 \end{pmatrix}.$$

The diagonal 1s confirm the unit lengths. The off-diagonal entries are 0.6, so the vectors are not perpendicular. Therefore $C^\top C \neq I_2$: this basis is not orthonormal, even though both vectors have length 1.

In the standard basis, the weights equal the entries of the observation: $w = x$. For a different orthonormal basis, the same observation can have different weights. The test $C^\top C = I_p$ checks the geometry of the basis; the chosen basis determines the weights.

2.12 Coordinates in an orthonormal basis

We now know how to check a basis. How do we find the weights needed to represent a point in that basis? For an orthonormal basis, a dot product with each basis vector gives the answer.

We know the observation x and choose the basis vectors $c_1, \ldots, c_p$. We want to find their weights $w_1, \ldots, w_p$ so that their weighted sum gives x. Put the basis vectors in the columns of C and the weights in w. Matrix multiplication performs that sum:

$$Cw = w_1 c_1 + \cdots + w_p c_p = x.$$

Thus $x = Cw$ is the compact form of a linear combination. Multiplying both sides by $C^\top$ gives

$$C^\top x = C^\top C w = I_p w = w.$$

Therefore the coordinates are

$$\boxed{w = C^\top x, \qquad w_j = c_j^\top x.}$$

The weight w_j tells us how far to move along the unit vector c_j. A positive weight follows its direction; a negative weight goes in the opposite direction. Perpendicular basis vectors do not contribute to this dot product, so it picks out just the weight we want.

For every observation $x^{(i)}$, the same calculation gives $w^{(i)} = C^\top x^{(i)}$. With a complete orthonormal basis, combining these weights with the basis vectors gives the original observation:

$$Cw^{(i)} = CC^\top x^{(i)} = x^{(i)}.$$

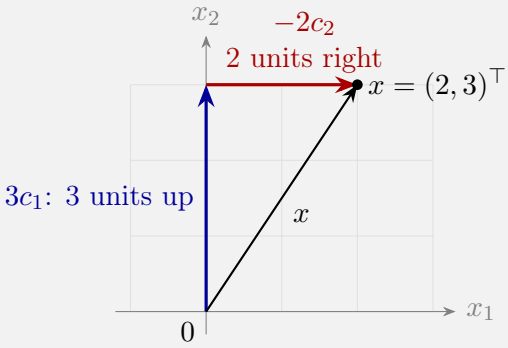
Here C is square and its columns form a basis of all of $\mathbb{R}^p$, so $CC^\top = I_p$ as well as $C^\top C = I_p$.

Example 10: Finding the weights of the point $(2, 3)$

Our observation is $x = (2, 3)^\top$: two units to the right and three units up. We want to describe this same point using two different unit directions:

$$c_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad \text{(up)}, \qquad c_2 = \begin{pmatrix} -1 \\ 0 \end{pmatrix} \quad \text{(left)}.$$

These vectors are perpendicular and have length 1, so they form an orthonormal basis. We seek weights w_1, w_2 for which $x = w_1 c_1 + w_2 c_2$.



Read the two moves in the drawing:

- Along c_1, we need to go up three units, so $w_1 = 3$.
- c_2 points left, but we need to go right two units. We move against its direction, so $w_2 = -2$.

The dot-product rule gives exactly these weights:

$$w_1 = c_1^\top x = 0 \cdot 2 + 1 \cdot 3 = 3,$$
$$w_2 = c_2^\top x = (-1) \cdot 2 + 0 \cdot 3 = -2.$$

Thus $w = (3, -2)^\top$. To check these weights, add their contributions:

$$Cw = 3c_1 - 2c_2 = \begin{pmatrix} 0 \\ 3 \end{pmatrix} + \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \end{pmatrix} = x.$$

The sum gives our original observation. Its weights are $(2, 3)$ in the standard basis and $(3, -2)$ in the chosen basis. Both sets of weights describe the same point using their respective basis vectors.

2.13 Nonstandard bases

The standard basis is convenient, but we can choose other directions. They must be linearly independent and span the space; they do not have to be perpendicular or have unit length.

Consider the basis from the slides:

$$c_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

The vectors are independent, but $c_1^\top c_2 = 4$ and each has length $\sqrt{5}$. This basis is not orthonormal.

To find the weights, we must still solve $Cw = x$.

Example 11: Representing $(5, 5)$ in a different basis

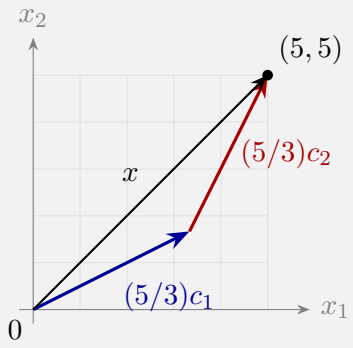
For $x = (5, 5)^\top$, the equation $w_1 c_1 + w_2 c_2 = x$ becomes

$$\begin{cases} 2w_1 + w_2 = 5, \\ w_1 + 2w_2 = 5. \end{cases}$$

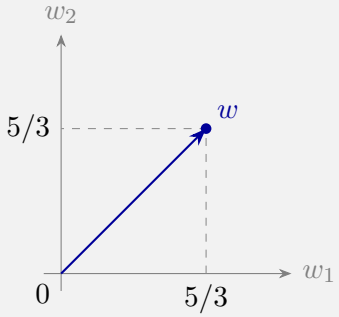
Solving the system gives $w_1 = w_2$, so

$$w_1 = w_2 = \frac{5}{3}, \qquad x = \frac{5}{3}c_1 + \frac{5}{3}c_2.$$

Original coordinates



New coordinates



On the left, the two contributions add up to the original point. On the right, we record their weights: $w = (5/3, 5/3)^\top$.

2.13.1 Encoding and decoding

Here we use the same basis as in the previous example. Its two vectors are the columns of the matrix C:

$$c_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

The vector $w = (w_1, w_2)^\top$ stores the two weights: how much of each basis vector we need to add. Multiplying C by w performs this sum:

$$Cw = w_1 c_1 + w_2 c_2.$$

Thus Cw is a point expressed in the original coordinates.

Encoding means starting from a point x and finding the weights w such that $Cw = x$. *Decoding* means starting from the weights w and computing Cw to obtain the point in its original coordinates.

For example, keep this same matrix C and choose the weights $w = (-2, 1)^\top$. Decoding gives

$$Cw = -2c_1 + c_2 = -2 \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} -3 \\ 0 \end{pmatrix}.$$

These weights describe the point $x = (-3, 0)^\top$. The weight -2 means taking twice c_1 in the opposite direction; the weight 1 means adding one copy of c_2.

Because these vectors form a basis, each point has exactly one pair of weights. Encoding a point and then decoding its weights returns that same point.

2.14 Finding the weights by least squares

We know the point x and the basis matrix C. We want to find the weights w with which to combine the columns of C. Each choice of weights produces a point Cw. Our question is: *which weights make this point as close as possible to x?*

Start with the basis and point from the previous example:

$$C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad x = \begin{pmatrix} 5 \\ 5 \end{pmatrix}.$$

Suppose we try the weights $w = (1, 1)^\top$. Adding one copy of each basis vector gives

$$Cw = \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \qquad Cw - x = \begin{pmatrix} -2 \\ -2 \end{pmatrix}.$$

We wanted $(5, 5)$, but obtained $(3, 3)$. The difference $Cw - x$ is the *reconstruction error*: how far off we are on each coordinate. To measure it with a single number, square each coordinate error and add the results. For these weights, the result is

$$(-2)^2 + (-2)^2 = 8.$$

Squaring makes each contribution nonnegative, so errors on different coordinates cannot cancel each other.

For a point with p coordinates, we call this sum $h(w)$:

$$h(w) = \|Cw - x\|_2^2 = \sum_{j=1}^{p} \big((Cw)_j - x_j\big)^2.$$

Here $(Cw)_j$ is the jth coordinate of the reconstructed point and x_j is the corresponding coordinate of the original point. *Least squares* means choosing the weights that make $h(w)$ as small as possible.

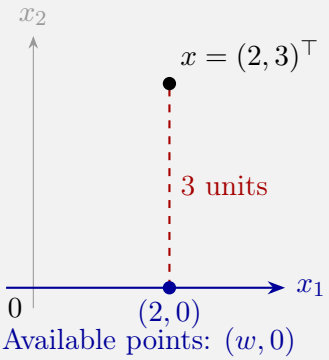
With a complete basis, we can reconstruct x exactly, so the minimum error is zero. We will obtain the same weights as by solving $Cw = x$. This formulation will also let us find the best approximation when we keep fewer directions and exact reconstruction is impossible.

Example 12: Approximating a point with one direction

Take the point $x = (2, 3)^\top$: two units to the right and three units up. Suppose we have only one direction: the horizontal vector $c_1 = (1, 0)^\top$. The matrix C now has one column, and the weight w is a single number:

$$C = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad Cw = \begin{pmatrix} w \\ 0 \end{pmatrix}.$$

Every point we can reconstruct lies on the horizontal axis: its second coordinate is always zero.



The point x is three units above the horizontal axis, so we cannot reach it exactly. The closest available point is $(2, 0)$, directly below x, obtained by choosing $w = 2$.

The squared reconstruction error is

$$h(w) = (w - 2)^2 + (0 - 3)^2 = (w - 2)^2 + 9.$$

Choosing $w = 2$ makes the first term zero. The minimum squared error is therefore 9.

We can also find the best weight by setting the derivative of h to zero:

$$h'(w) = 2(w - 2) = 0 \quad \Longrightarrow \quad w = 2.$$

2.14.1 The normal equations

In the previous example, we found the best value of a single weight w by solving $h'(w) = 0$. We now generalise this procedure from one weight to several weights, finding their best values together. This will give one equation for each weight.

If C has K columns, we need K weights $w_1, \ldots, w_K$. With a complete basis, $K = p$; with fewer directions, $K < p$.

To find the weights that minimise h, we compute a derivative with respect to each weight. The derivative $\partial h/\partial w_j$ describes how the error changes when we change w_j slightly while keeping the other weights fixed.

The vector collecting all these derivatives is called the *gradient*, written $\nabla_w h$:

$$\nabla_w h(w) = \left(\frac{\partial h}{\partial w_1}, \ldots, \frac{\partial h}{\partial w_K} \right)^\top.$$

At a minimum, all these derivatives are zero. For this quadratic function, when the columns of C are linearly independent, setting the gradient to zero finds the unique minimum.

Recall the squared reconstruction error:

$$h(w) = \|Cw - x\|_2^2 = \sum_{j=1}^{p} \big((Cw)_j - x_j \big)^2.$$

Differentiating this function with respect to the weights gives

$$\nabla_w h(w) = 2C^\top (Cw - x) = 0.$$

Divide by 2 and expand the product:

$$C^\top (Cw - x) = 0 \quad \Longrightarrow \quad C^\top Cw - C^\top x = 0.$$

Moving the second term to the right gives a linear system called the *normal equations*:

Proposition 2.1: Normal equations

For a fixed matrix C with linearly independent columns, the unique weights that minimise $\|Cw - x\|_2^2$ satisfy

$$\boxed{C^\top Cw = C^\top x.}$$

The matrix $C^\top C$ and the vector $C^\top x$ are known, because we know C and x. We solve this system for the unknown weights w. The matrix $C^\top C$ accounts for the lengths of the chosen vectors and the dot products between them. If the columns are orthonormal, it equals the identity and the equations reduce to $w = C^\top x$.

Example 13: Solving the normal equations

Take the basis vectors $c_1 = (2, 1)^\top$ and $c_2 = (1, 2)^\top$, and the point $x = (5, 5)^\top$. The columns of C are c_1 and c_2, in that order. The known quantities are

$$C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad x = \begin{pmatrix} 5 \\ 5 \end{pmatrix},$$

$$C^\top C = \begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix}, \quad C^\top x = \begin{pmatrix} 15 \\ 15 \end{pmatrix}.$$

The matrix equation $C^\top C w = C^\top x$ becomes

$$\begin{cases} 5w_1 + 4w_2 = 15, \\ 4w_1 + 5w_2 = 15. \end{cases}$$

Solving gives $w_1 = w_2 = 5/3$. With these weights, $Cw = (5, 5)^\top = x$, so $h(w) = 0$: we reconstruct the point exactly.

2.14.2 Encoding the whole dataset

In the previous example, we found the weights of one point. A dataset is a collection of points: we now want to describe every one of them using weights in the same chosen directions.

This prepares us to describe data with fewer numbers. With a complete basis, each point still has as many weights as original coordinates. In the next section, we will choose fewer directions and use fewer weights per point. When these directions cannot reconstruct a point exactly, we will find its best approximation.

For a concrete example, choose the two basis vectors below and put them in the columns of C:

$$c_1 = (2,1)^\top, \qquad c_2 = (1,2)^\top, \qquad C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

Suppose the dataset contains just two points:

Original point	**Combination of the directions**	**Weights**
$(3,3)^\top$	$c_1 + c_2$	$(1,1)^\top$
$(6,6)^\top$	$2c_1 + 2c_2$	$(2,2)^\top$

The first point uses each basis vector once; the second uses each twice. *Encoding the whole dataset* means finding these weights for every observation. The matrix C is shared by all points; each point has its own weights.

These same points could also be described using just $d = (1,1)^\top$: the first is $3d$, the second $6d$. We would need only one weight per point. This is the benefit we will explore when choosing fewer directions.

For n observations, write $x^{(i)}$ for point i and $w^{(i)}$ for its weight vector. Find the weights one point at a time by solving the same normal equations as before:

$$C^\top C w^{(i)} = C^\top x^{(i)}, \qquad i = 1, \ldots, n.$$

Let us carry out this calculation for both points in the table. Each weight vector has two entries, $w_1^{(i)}$ and $w_2^{(i)}$, one for each column of C. The matrix on the left is the same for both points, so we compute it once:

$$C^\top C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix}.$$

First point: $x^{(1)} = (3, 3)^\top$**.** Compute the right-hand side using this point:

$$C^\top x^{(1)} = \begin{pmatrix} 2 \cdot 3 + 1 \cdot 3 \\ 1 \cdot 3 + 2 \cdot 3 \end{pmatrix} = \begin{pmatrix} 9 \\ 9 \end{pmatrix}.$$

The normal equations are therefore

$$\begin{cases} 5w_1^{(1)} + 4w_2^{(1)} = 9, \\ 4w_1^{(1)} + 5w_2^{(1)} = 9. \end{cases}$$

To solve the system, subtract the second equation from the first. This gives $w_1^{(1)} - w_2^{(1)} = 0$, so the weights are equal. Substituting into the first equation gives $9w_1^{(1)} = 9$, hence $w^{(1)} = (1, 1)^\top$. These weights reconstruct the first point: $Cw^{(1)} = (3, 3)^\top$.

Second point: $x^{(2)} = (6, 6)^\top$**.** Keep the same $C^\top C$, and compute the right-hand side for this point:

$$C^\top x^{(2)} = \begin{pmatrix} 2 \cdot 6 + 1 \cdot 6 \\ 1 \cdot 6 + 2 \cdot 6 \end{pmatrix} = \begin{pmatrix} 18 \\ 18 \end{pmatrix}.$$

This gives a separate system for the second point's weights:

$$\begin{cases} 5w_1^{(2)} + 4w_2^{(2)} = 18, \\ 4w_1^{(2)} + 5w_2^{(2)} = 18. \end{cases}$$

Solving in the same way gives $w_1^{(2)} - w_2^{(2)} = 0$. The first equation then becomes $9w_1^{(2)} = 18$, hence $w^{(2)} = (2, 2)^\top$. These weights reconstruct the second point: $Cw^{(2)} = (6, 6)^\top$.

Multiplying $Cw^{(i)}$ reconstructs point i from its weights. To assess all the reconstructions together, add their squared errors and divide by the number of observations:

$$\text{average reconstruction error} = \frac{1}{n} \sum_{i=1}^{n} \|Cw^{(i)} - x^{(i)}\|_2^2.$$

Each term is the error for one observation. Changing that observation's weights affects only its own term, so finding the best weights for each point separately also minimises the average error.

2.15 Using fewer directions: subspaces

A basis of $\mathbb{R}^p$ has exactly p linearly independent vectors. Using more than p vectors can still give a spanning set, but those vectors cannot all be independent. Using fewer than p independent vectors gives a different possibility: we can represent only part of the original space.

Let $K < p$ be the number of directions we keep. The vectors $c_1, \ldots, c_K$ still have p entries; we now place them in a rectangular matrix

$$C = [\, c_1 \ \cdots \ c_K \,] \in \mathbb{R}^{p \times K}.$$

Combining its columns gives a *subspace* of $\mathbb{R}^p$.

Definition 2.11: Subspace spanned by the columns

The space generated by $c_1, \ldots, c_K$ is the set of all their linear combinations:

$$S = \operatorname{span}\{c_1, \ldots, c_K\} = \{Cw : w \in \mathbb{R}^K\}.$$

If the columns are linearly independent, S has dimension K and they form a basis of S. This set contains the origin and is closed under vector addition and multiplication by scalars.

A subspace of dimension $p - 1$ is called a *hyperplane*.

In two dimensions, keeping one direction gives a line through the origin. In three dimensions, keeping two independent directions gives a plane through the origin.

Example 14: A line in a two-dimensional space

Suppose we want to describe points in the plane with a single weight w. We choose the diagonal direction $c_1 = (1, 1)^\top$. The matrix C has only this column, so $C = c_1$. Multiplying by w gives

$$Cw = w \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} w \\ w \end{pmatrix}.$$

Every combination has equal coordinates: the available points lie on the line $x_2 = x_1$. We can represent $(3, 3)^\top$ exactly with $w = 3$. But no value of w gives $(2, 0)^\top$: its first coordinate would require $w = 2$ and its second would require $w = 0$.

We now store just K weights for an observation that originally had p coordinates. This is dimensionality reduction. It can be useful when the observations lie on, or close to, a smaller subspace. An observation outside that subspace cannot be recovered exactly; next we find its closest available approximation.

2.16 Projection onto a subspace

Suppose we want to represent $x = (2, 0)^\top$ using only the direction $(1, 1)^\top$. Every weight produces $(w, w)^\top$, so no weight reconstructs x exactly. We therefore choose the weight that gives the closest point on the line. This replaces exact reconstruction with the best approximation available in the chosen directions.

More generally, C contains K chosen directions for points with p original coordinates. Their combinations Cw form the subspace S. We write w_* for the weights that minimise the squared error, and $\hat{x} = Cw_*$ for the resulting reconstruction. The star marks the best weights; the hat distinguishes the reconstruction from the original point x.

Definition 2.12: Orthogonal projection

Let the columns of $C \in \mathbb{R}^{p \times K}$ be linearly independent and span S. The orthogonal projection of x onto S is

$$\hat{x} = Cw_*, \qquad w_* = \arg\min_{w \in \mathbb{R}^K} \|Cw - x\|_2^2.$$

Thus $\hat{x}$ is the point of S with the smallest Euclidean distance from x; w_* contains its coordinates in the chosen basis of S. The notation $\arg\min$ means "the weights that make this error smallest"; it returns a weight vector, rather than the value of the minimum error.

To find w_*, solve the normal equations:

$$C^\top C w_* = C^\top x.$$

These equations minimise the same squared error as before, even when the minimum is positive. They find the closest reconstruction without requiring $Cw_* = x$.

There is also a compact formula for the solution. Since the columns of C are independent, $C^\top C$ has an inverse, written $(C^\top C)^{-1}$. An inverse undoes multiplication by a matrix: $(C^\top C)^{-1}(C^\top C) = I_K$. Multiplying both sides of the normal equations by this inverse leaves w_* on the left:

$$(C^\top C)^{-1}(C^\top C) w_* = (C^\top C)^{-1} C^\top x.$$

Thus the weights and reconstruction can also be written as

$$w_* = (C^\top C)^{-1} C^\top x, \qquad \hat{x} = C(C^\top C)^{-1} C^\top x.$$

Example 15: The closest point on the line $x_2 = x_1$

We want to approximate $x = (2, 0)^\top$ using the diagonal direction $c_1 = (1, 1)^\top$. Here $C = c_1$ has one column, so we need just one weight. Compute the two quantities in the normal equations:

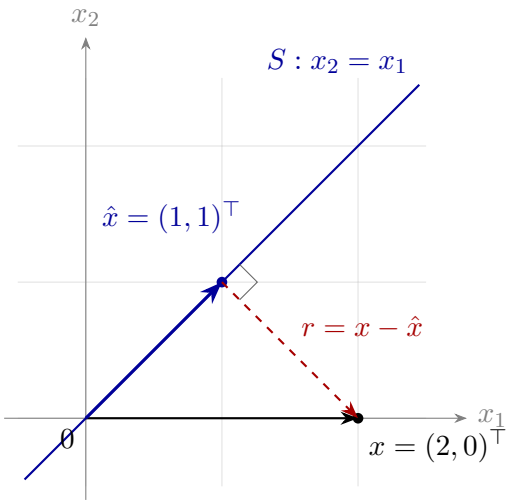
$$C^\top C = 1 \cdot 1 + 1 \cdot 1 = 2, \qquad C^\top x = 1 \cdot 2 + 1 \cdot 0 = 2.$$

The equation $C^\top C w_* = C^\top x$ becomes $2w_* = 2$, so $w_* = 1$. Multiplying the chosen direction by this weight gives

$$\hat{x} = Cw_* = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

We cannot reach $(2, 0)$ on this line, but $(1, 1)$ is the closest point. The vector left over, called the residual, is $r = x - \hat{x} = (1, -1)^\top$. It records an error of 1 in the first coordinate and -1 in the second. Its squared length is $1^2 + (-1)^2 = 2$, the minimum squared reconstruction error.

The diagram shows the same example: $C = (1, 1)^\top$, $x = (2, 0)^\top$, and the closest reconstruction $\hat{x} = (1, 1)^\top$. The blue line is S: all points of the form $(w, w)^\top$. The dashed segment is the residual $r = x - \hat{x}$ from the reconstruction to the original point.



The dashed segment meets the blue line at a right angle. Its length measures the distance between the original point and its reconstruction; its squared length is the reconstruction error.

2.16.1 Why the projection is perpendicular

For any point, the *residual* is $r = x - \hat{x} = x - Cw_*$. It is the vector from the reconstruction to the original point. Using the normal equations gives

$$C^\top r = C^\top x - C^\top C w_* = 0.$$

Row j of $C^\top$ is $c_j^\top$, so entry j of this equation is $c_j^\top r = 0$. This is a zero dot product: the residual is perpendicular to each chosen direction and to every combination of them. In the diagonal-line example, this check is

$$(1, 1)\begin{pmatrix} 1 \\ -1 \end{pmatrix} = 1 - 1 = 0.$$

Any other point on the line adds a sideways distance to this perpendicular distance. By Pythagoras, its squared distance to x is larger. This explains why the perpendicular projection is the closest point.

2.16.2 Encoding and decoding the projection

With a fixed C, the two steps are:

Step	What we do
Encoding	Find the K weights w_* that best represent x.
Decoding	Compute $\hat{x} = Cw_*$, returning to p coordinates.

For this example, encoding means solving $2w_* = 2$ to store the single weight 1. Decoding means multiplying $C = (1, 1)^\top$ by that weight:

$$\underbrace{\begin{pmatrix} 2 \\ 0 \end{pmatrix}}_{\text{original: 2 coordinates}} \xrightarrow{\text{encode}} \underbrace{1}_{\text{1 weight}} \xrightarrow{\text{decode}} \underbrace{\begin{pmatrix} 1 \\ 1 \end{pmatrix}}_{\text{reconstructed: 2 coordinates}}.$$

Both $(2, 0)^\top$ and $(1, 1)^\top$ have this same encoding, 1. The weight alone cannot tell us which of those points we started with. Decoding returns two coordinates on the line and leaves out the residual. If the original point already lies on the line, that residual is zero and the reconstruction is exact.

2.17 Orthonormal directions in a subspace

We still want the closest reconstruction in a chosen subspace. We can simplify the calculation by using K perpendicular unit vectors as its basis. Put these vectors in the columns of C; each has p coordinates, so C has p rows and K columns. The diagonal entries of $C^\top C$ are their squared lengths, all 1; the other entries are dot products between different vectors, all 0. Therefore

$$C^\top C = I_K.$$

Substituting this identity into $C^\top C w_* = C^\top x$ gives $I_K w_* = C^\top x$, hence $w_* = C^\top x$. We then combine the directions with these weights to reconstruct the point:

$$\boxed{w_* = C^\top x, \qquad \hat{x} = C w_* = C C^\top x.}$$

Weight j is the dot product $c_j^\top x$: the signed amount of x along that unit direction. Multiplying each direction by its weight and adding the results gives the projection $\hat{x}$.

Example 16: Normalising the direction of the same line

We again approximate $x = (2, 0)^\top$ on the line $x_2 = x_1$. The direction $(1, 1)^\top$ has length $\sqrt{1^2 + 1^2} = \sqrt{2}$. To make it a unit vector, divide each entry by this length:

$$c_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad C = c_1.$$

The normalised vector still points along the same line. Its unit length lets us find the weight by a dot product:

$$w_* = c_1^\top x = \frac{1}{\sqrt{2}} \cdot 2 + \frac{1}{\sqrt{2}} \cdot 0 = \sqrt{2}.$$

Decoding multiplies this weight by the new direction:

$$\hat{x} = \sqrt{2} c_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Previously the direction was $(1, 1)^\top$ and its weight was 1. After making it a unit vector, the weight becomes $\sqrt{2}$. The line and the projected point stay the same.

Here C is 2×1. Thus $C^\top C$ is 1×1, whereas $CC^\top$ is 2×2. Their entries are

$$C^\top C = \frac{1}{2} + \frac{1}{2} = 1, \qquad CC^\top = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

The first checks the unit length of our one direction. The second combines encoding and decoding into a single matrix multiplication, $CC^\top x = C(C^\top x)$. For our point,

$$CC^\top \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

With orthonormal columns, $CC^\top$ is called a projection matrix: multiplying a point by it returns its closest point in S. For $K < p$, it keeps the part along the chosen directions and leaves out the perpendicular residual. For $K = p$, a complete orthonormal basis gives $CC^\top = I_p$, so every point is recovered exactly.

2.17.1 From three coordinates to two

Suppose an observation has three coordinates, but we want to describe it using two weights. Choose two independent directions in three-dimensional space; their combinations form a plane. Encoding finds the two weights, and decoding returns the closest point on that plane.

Example 17: Projection onto the coordinate plane

We want to approximate $x = (2, 3, 4)^\top$ using only directions along the first two coordinate axes. Choose the perpendicular unit vectors

$$c_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \qquad C = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}.$$

Every combination of these vectors has third coordinate 0, so they span the plane $x_3 = 0$. Since $C^\top C = I_2$, encoding gives

$$w_* = C^\top x = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}.$$

The weights tell us to use c_1 twice and c_2 three times. Decoding and computing the residual give

$$\hat{x} = Cw_* = 2c_1 + 3c_2 = \begin{pmatrix} 2 \\ 3 \\ 0 \end{pmatrix}, \qquad r = x - \hat{x} = \begin{pmatrix} 0 \\ 0 \\ 4 \end{pmatrix}.$$

The first two coordinates are preserved. The third is lost because the chosen directions cannot move us out of the plane. The residual is perpendicular to that plane, and its squared length is $4^2 = 16$.

2.17.2 What changes when we reduce the dimension?

For independent columns, the same normal equations work in every case:

$$C^\top C w_* = C^\top x, \qquad \hat{x} = C w_*.$$

What changes is how many directions we keep and whether they are orthonormal:

Directions	**Finding the weights**	**Reconstruction**
Complete basis	Solve the normal equations.	Always $\hat{x} = x$.
Complete, orthonormal	$w_* = C^\top x$.	Always $\hat{x} = x$.
Basis of a subspace	Solve the normal equations.	Closest point in S.
Orthonormal in a subspace	$w_* = C^\top x$.	Closest point in S.

With fewer directions, reconstruction is exact precisely when the observation already lies in their subspace. Otherwise we retain its projection and lose its perpendicular component. The average squared length of these residuals is the dataset's reconstruction error.

Source: Spinelli, Lecture 01 (data_vectors), slides 18–35. The slide deck credits Machine Learning Refined, Chapter 8.

3

Principal Component Analysis

4

Optimization

Start of lecture 03 (Spinelli)

In the chapter on Data Representation, we chose weights to make a reconstruction error as small as possible. Machine learning uses the same idea: a loss function scores the error for each choice of a model's weights, and minimizing it selects the weights with the best fit according to that score. We now study how to minimize or maximize a function, beginning with one adjustable number w.

4.1 What are we trying to optimize?

Let $g(w)$ assign a numerical score to the choice w. We vary w to find the smallest or largest value of g, depending on the task.

Definition 4.1: Objective function and optimization

The *objective function* is the function whose value we want to minimize or maximize. For an objective $g(w)$, we write

$$\min_w g(w) \qquad \text{or} \qquad \max_w g(w).$$

The subscript w identifies the input we are allowed to change. When we minimize an error or a cost, the objective is also called a *loss function* or a *cost function*.

We use w_* for an input that achieves the desired optimum. Thus w_* is the chosen input, and $g(w_*)$ is its score. For minimization, $w_* = \arg\min_w g(w)$ means "choose an input at which g is smallest". For maximization, $\arg\max$ has the corresponding meaning with "largest". More than one input may achieve the same best score.

4.2 Global minima and maxima

A global optimum must be best among *all* the inputs we are considering. In the following examples, w can be any real number.

Definition 4.2: Global minimum and global maximum

An input w_* is a *global minimum point* of g if

$$g(w_*) \leq g(w) \qquad \text{for every } w.$$

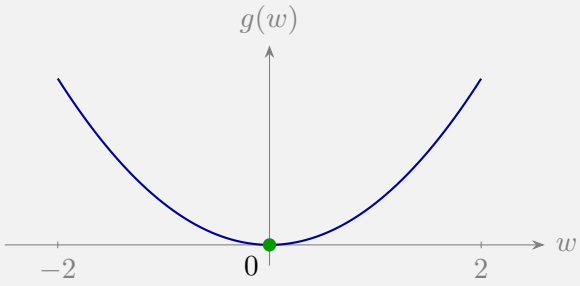
It is a *global maximum point* if

$$g(w_*) \geq g(w) \qquad \text{for every } w.$$

The values $g(w_*)$ are called the global minimum value and the global maximum value, respectively.

Example 18: The bottom of a parabola

Suppose we want to choose w to minimize $g(w) = w^2$. The horizontal axis shows the input w; the vertical axis shows its score $g(w)$. The green dot marks the bottom of the curve.



For example, $g(-2) = 4$, $g(0) = 0$, and $g(2) = 4$. Every square is nonnegative, so $g(0) = 0 \leq w^2$ for every w. Therefore $w_* = 0$ is the global minimum point, and the minimum value is 0.

If we turn the curve upside down, the function becomes $-g(w) = -w^2$. Every value is now at most 0, so the same input $w_* = 0$ is its global maximum point.

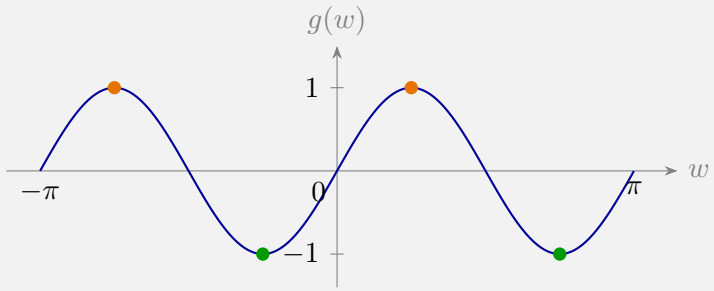
This change of sign works for any objective. If one input gives a larger value of g, it gives a smaller value of $-g$. We can therefore solve a maximization problem by minimizing the negative function:

$$\arg\max_w g(w) = \arg\min_w \big(-g(w)\big), \qquad \max_w g(w) = -\min_w \big(-g(w)\big).$$

The first equality identifies the optimizing inputs; the second relates the optimum values.

Example 19: Several inputs can be globally optimal

Consider the objective $g(w) = \sin(2w)$. Its graph repeatedly reaches the same lowest height, -1, and the same highest height, 1. Green dots mark minima; orange dots mark maxima.

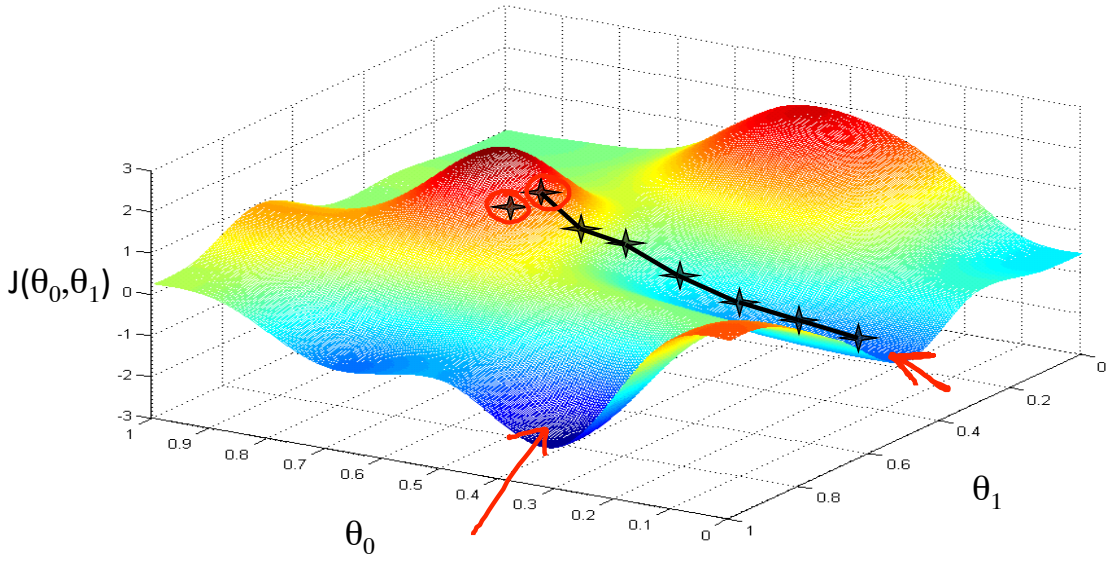


For instance, $w = -\pi/4$ and $w = 3\pi/4$ both give $g(w) = -1$. No input can produce a smaller value, so both are global minimum points. Likewise, $w = -3\pi/4$ and $w = \pi/4$ give the global maximum value 1.

The sine curve repeats, so there are infinitely many global minimum points and infinitely many global maximum points. An optimum need not be unique.

4.3 Local minima and maxima

Now imagine a curve with several valleys of different depths. The bottom of one valley is lower than the points immediately around it, even when another valley is deeper. This motivates a comparison with *nearby* inputs.



A function with several valleys. Source: Andrew Ng, Machine Learning, Lecture 2, slide 25.

Here the function has two inputs: the adjustable parameters θ_0 and θ_1 on the horizontal axes. The height $J(\theta_0, \theta_1)$ is the value we want to minimise; J plays the same role as g in our notation. The black path shows successive downhill steps into the valley on the right. The valley in front is deeper: being lower than nearby points does not necessarily mean having the lowest value overall.

Definition 4.3: Local minimum and local maximum

An input w_* is a *local minimum point* if

$$g(w_*) \leq g(w) \qquad \text{for all } w \text{ sufficiently near } w_*.$$

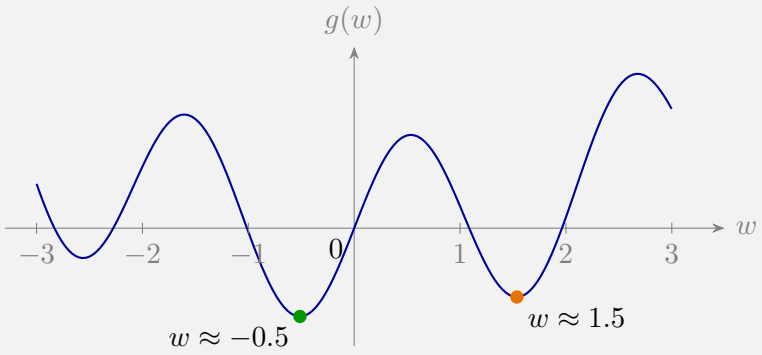
It is a *local maximum point* if the inequality is reversed. Here "sufficiently near" means that the comparison holds throughout some small interval around w_*.

Example 20: Valleys of different depths

Consider the sum of an oscillating function and a quadratic term:

$$g(w) = \sin(3w) + 0.1w^2.$$

The sine term produces peaks and valleys. The quadratic term grows as $|w|$ increases, so these valleys have different depths. The graph shows two of them with colored dots.



The green point near $w = -0.5$ is the deepest valley: its value is about -0.973. It is a global minimum point. The orange point near $w = 1.5$ is also lower than its immediate neighbors, but its value is about -0.759. It is a local minimum point with a larger objective value.

The peaks near $w \approx 0.536$ and $w \approx 2.678$ are local maximum points: their values are larger than those at nearby inputs.

Imagine searching for a minimum by making small changes that lower g. In this example, the search can stop in the orange valley. Reaching the deeper green valley requires first climbing out of the orange one. The orange point is a local minimum, but the green point has a lower value of g.

4.4 Why these are called zero-order conditions

The global and local definitions compare values of g directly. They are called *zero-order conditions* because they use the function itself. First-order conditions use first derivatives, which describe the slope of the graph. Second-order conditions use second derivatives, which describe how that slope changes. The definitions tell us what it means to be a minimum or maximum; derivatives will help us find candidate optimum points without comparing every possible input individually.

Source: Spinelli, Lecture 03 (Optimization), slides 2–11.

Additional explanations: Watt, Borhani, and Katsaggelos, Machine Learning Refined (2016), Sections 1.4, 2.1.1, and 2.2.1.